

AutoML Framework Implementation

Internship Project Report — Task 2

1. Introduction

Selecting the right machine learning algorithm for a given dataset is often a repetitive, trial-and-error process. This project implements a lightweight AutoML (Automated Machine Learning) framework that automates this process end-to-end: it preprocesses a dataset, trains several candidate classification models, compares their performance, and automatically selects the best one — all through a single, reusable pipeline.

2. Objective

The objective is to build a generic AutoML engine that can automatically train multiple machine learning models on any tabular classification dataset and select the best performing model based on test accuracy, without requiring dataset-specific code.

3. Datasets Used

To demonstrate that the framework generalizes across different types of problems, it was tested on three datasets with different characteristics:

Dataset	Records	Features	Target	Notes
Diabetes	768	8 (numeric)	Outcome (binary)	Zero-as-missing values
Iris	150	4 (numeric)	species (3 classes)	Clean multi-class benchmark
Loan	491	12 (mixed)	Loan_Status (binary)	Categorical + real missing values

4. Framework Architecture

The framework follows a fixed pipeline for every dataset:

```
Input Dataset → Data Preprocessing → Train Multiple Models → Compare Accuracy →  
Select Best Model → Generate Report
```

Each dataset is registered in a configuration dictionary specifying its file path, target column, identifier columns to drop, and any columns where a value of 0 actually represents a missing measurement. The same preprocessing and training code then runs unmodified on any registered dataset.

5. Data Preprocessing

For each dataset, the framework automatically: (1) drops irrelevant identifier columns such as Loan_ID; (2) replaces implausible zero values with the column median for the Diabetes dataset; (3) imputes

missing values using the median for numeric columns and the mode for categorical columns, which handles real missing data present in the Loan dataset (Gender, Self_Employed, Credit_History, etc.); (4) one-hot encodes categorical predictors; (5) label-encodes non-numeric target columns; and (6) scales all numeric features using StandardScaler after a stratified 80/20 train-test split.

6. Models Compared

Five classification algorithms are trained and evaluated for every dataset: Logistic Regression, Random Forest (200 trees, max depth 8), Decision Tree (max depth 8), Support Vector Machine with an RBF kernel, and K-Nearest Neighbors (k=7). The model with the highest test-set accuracy is automatically selected as the best model for that dataset.

7. Results

Dataset	Best Model	Accuracy
Diabetes	Random Forest	74.7%
Iris	SVM	96.7%
Loan	Random Forest	83.8%

The Iris dataset achieved the highest accuracy (96.7%) as expected, since it is small, clean, and linearly well-separated. The Diabetes and Loan datasets, which involve more noise, class imbalance, and (for Loan) missing/categorical data, achieved more moderate accuracies of 74.7% and 83.8% respectively, with Random Forest performing best on both.

8. Visualizations

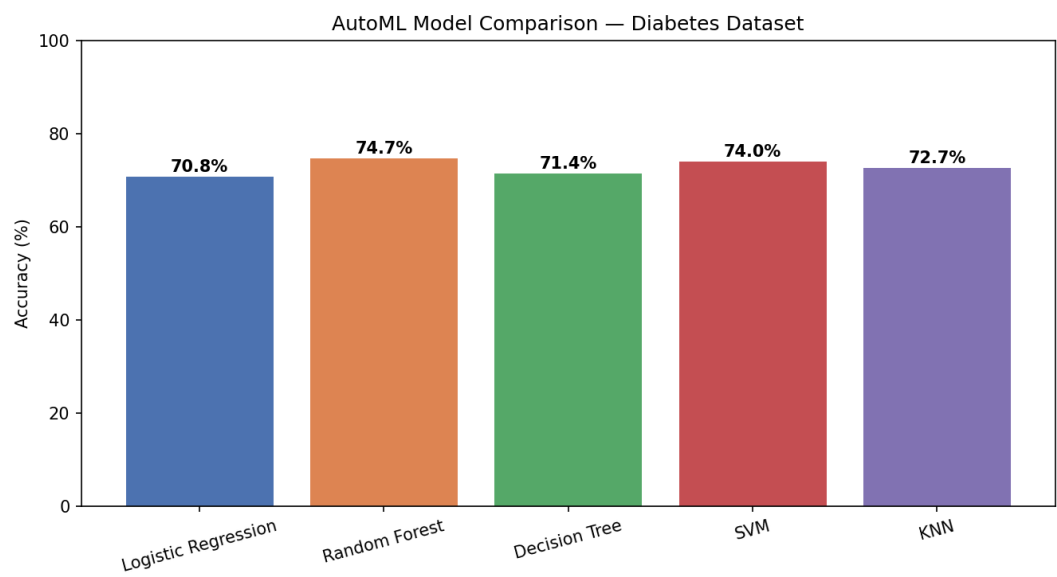


Figure 1: Model accuracy comparison on the Diabetes dataset.

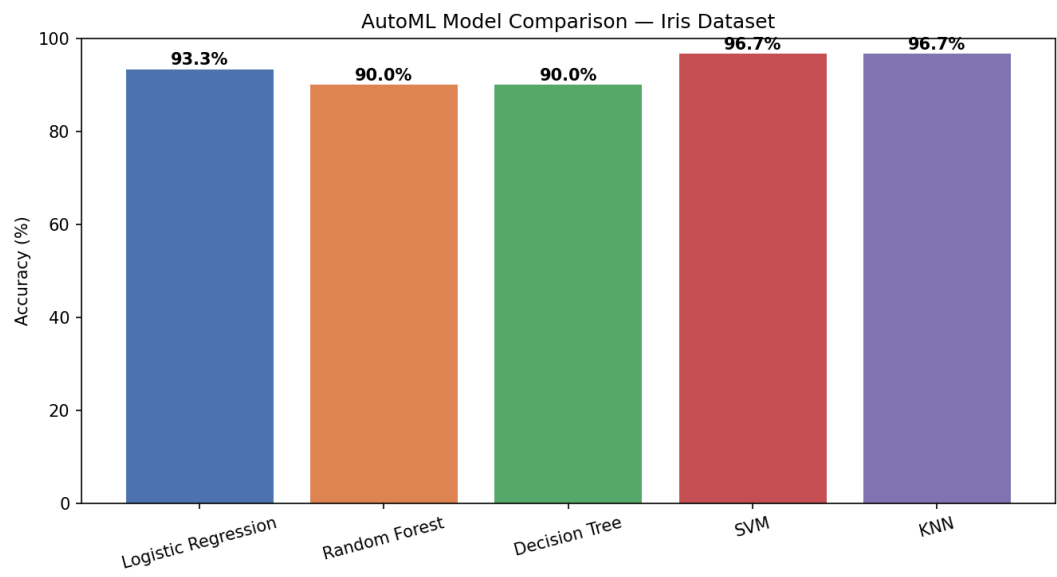


Figure 2: Model accuracy comparison on the Iris dataset.

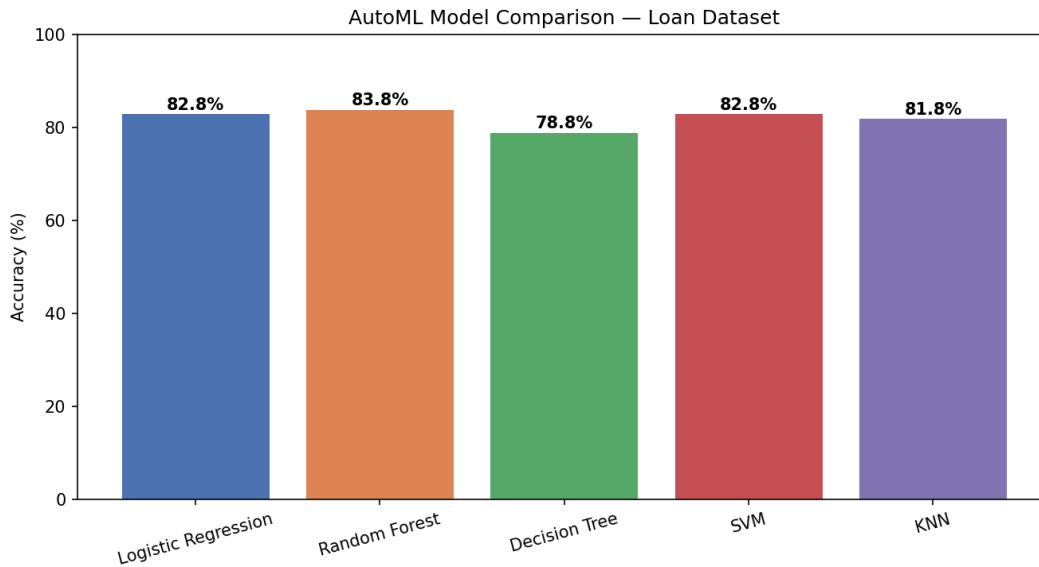


Figure 3: Model accuracy comparison on the Loan dataset.

9. Conclusion

This project demonstrates a reusable AutoML framework capable of automatically preprocessing, training, comparing, and selecting the best classification model across datasets with very different characteristics — purely numeric, small multi-class, and mixed categorical/numeric with missing data. The winning model, its fitted scaler, and feature metadata are saved per dataset in `outputs/`, and a consolidated report is generated automatically in both text and JSON form.

10. Future Improvements

Potential enhancements include hyperparameter tuning per model (grid/randomized search), cross-validation instead of a single train/test split, adding more candidate algorithms (e.g. Gradient Boosting, XGBoost), automatic handling of class imbalance, and a simple command-line interface (e.g. ``automl.py --dataset custom.csv --target label``) so any new CSV file can be run through the framework without editing the code.